

User Datagram Protocol

(UDP)

Outline

Introduction

User Datagram

UDP Services

UDP Application

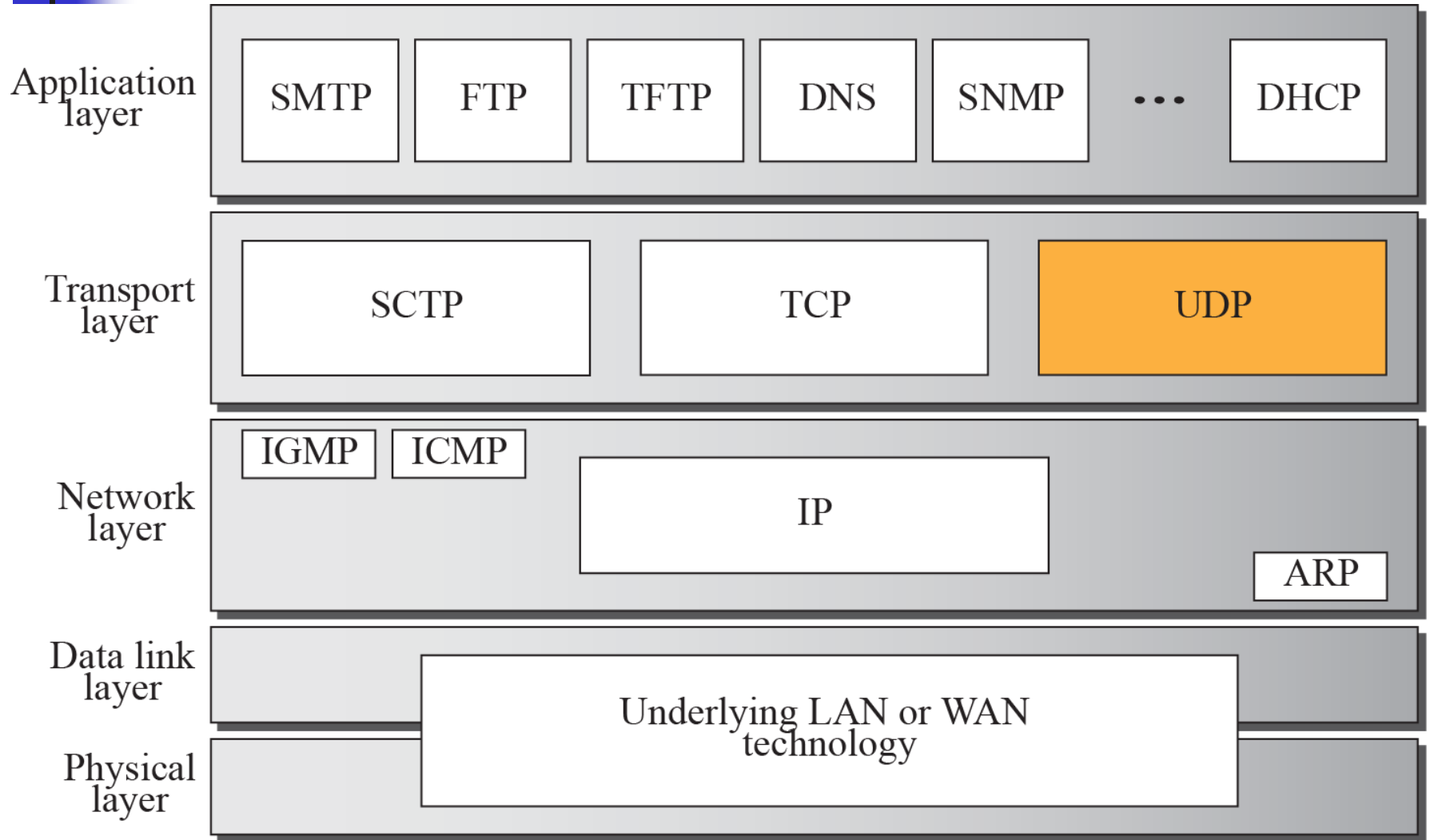
Transport layer protocol has several responsibilities

- To create process to process communication
- To provide control flow mechanism

**UDP is connectionless and unreliable protocol located at
Transport layer
(i.e. between the application layer and the IP layer)**

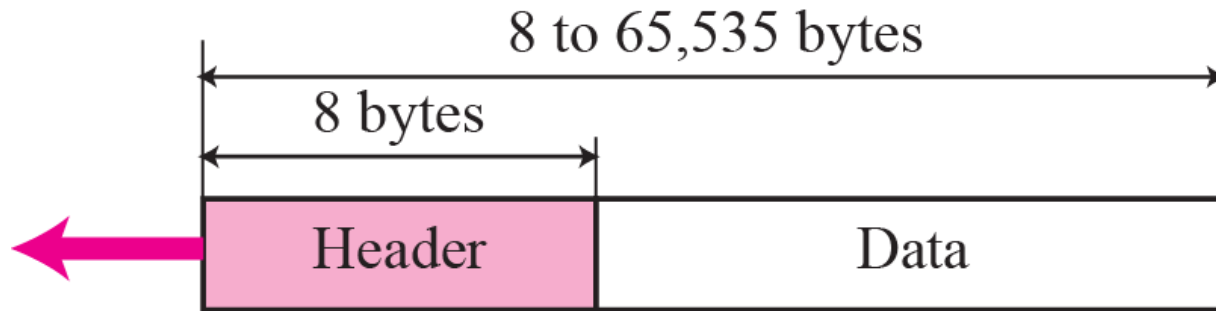
**serves as the intermediary between the application
programs and the network operations.**

Position of UDP in the TCP/IP protocol suite

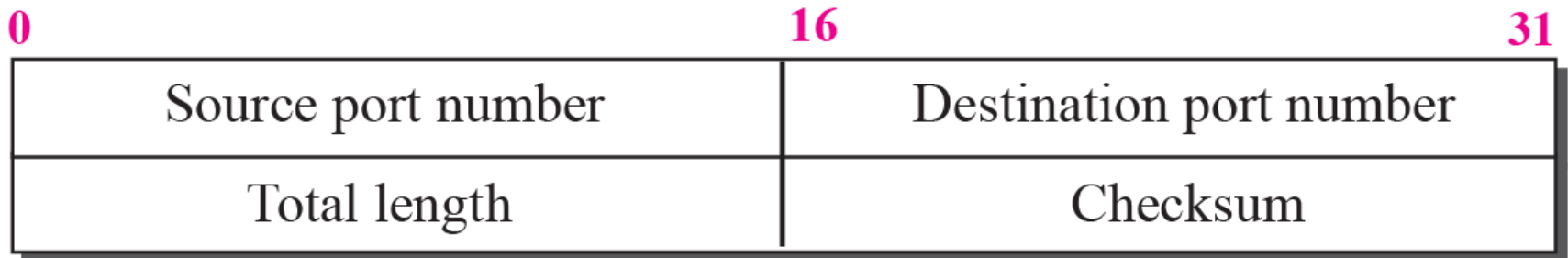


UDP packets, called user datagrams, have a fixed-size header of 8 bytes.

format of a user datagram.



a. UDP user datagram



b. Header format

Example 14.1

The following is a dump of a UDP header in hexadecimal format.

```
CB84000D001C001C
```

- a. What is the source port number?
- b. What is the destination port number?
- c. What is the total length of the user datagram?
- d. What is the length of the data?
- e. Is the packet directed from a client to a server or vice versa?

0	16	31
Source port number	Destination port number	
Total length	Checksum	

b. Header format

0	16	31
Source port number	Destination port number	
Total length	Checksum	

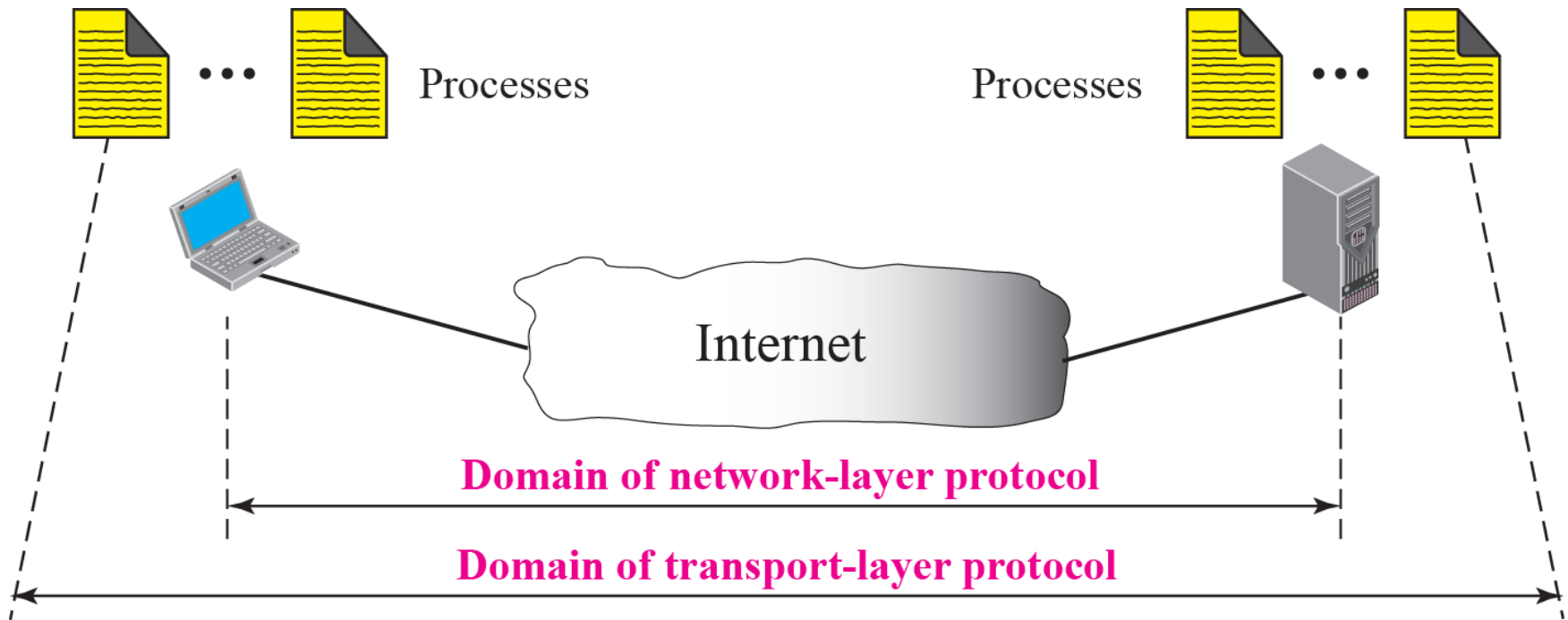
b. Header format

Solution

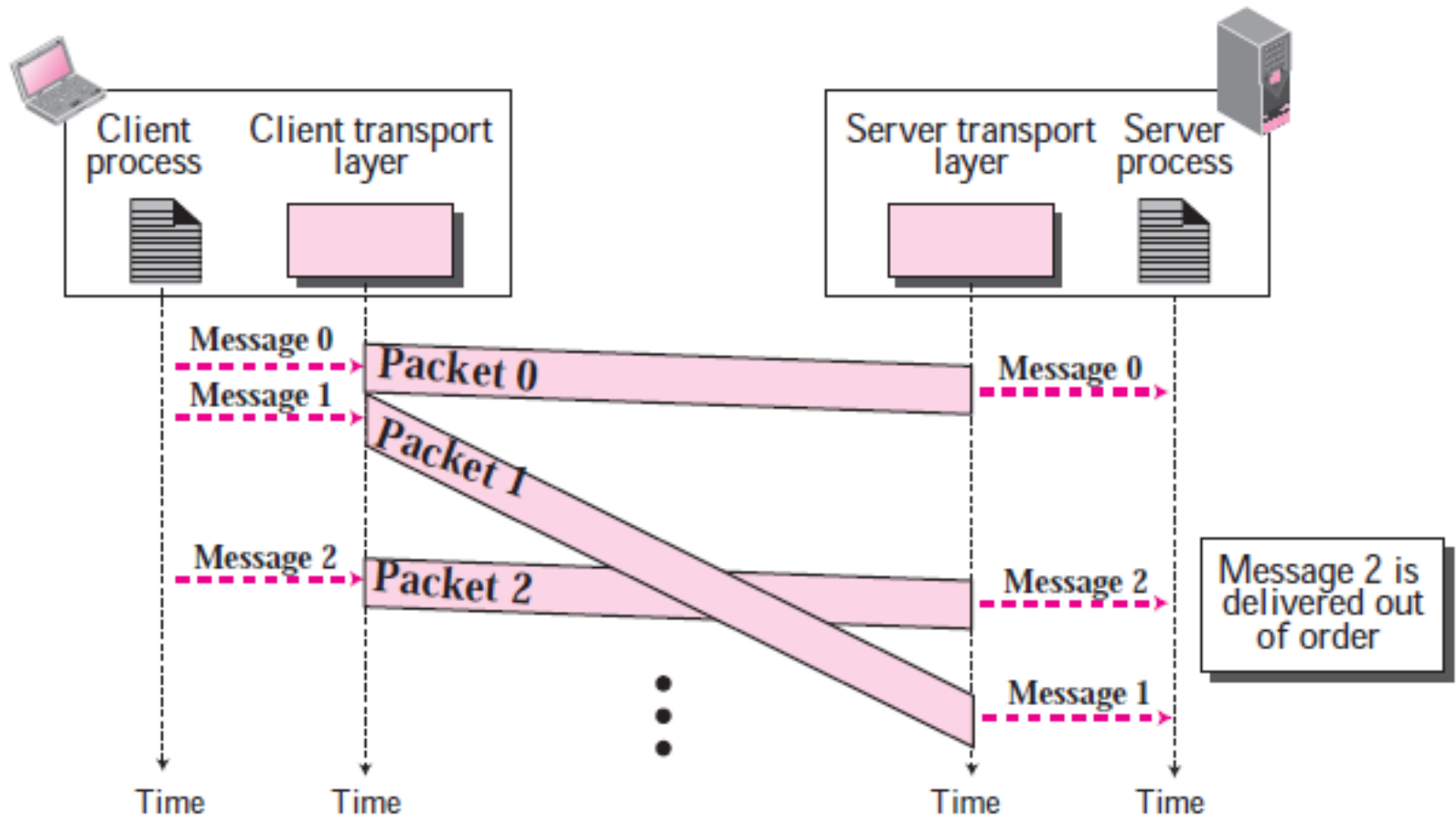
CB84000D001C001C

- a. The source port number is the first four hexadecimal digits $(CB84)_{16}$ or 52100.
- b. The destination port number is the second four hexadecimal digits $(000D)_{16}$ or 13.
- c. The third four hexadecimal digits $(001C)_{16}$ define the length of the whole UDP packet as 28 bytes.
- d. The length of the data is the length of the whole packet minus the length of the header, or $28 - 8 = 20$ bytes.
- e. Since the destination port number is 13 (well-known port), the packet is from the client to the server.

Network layer versus transport layer



Connectionless Service



Connection- Oriented Service

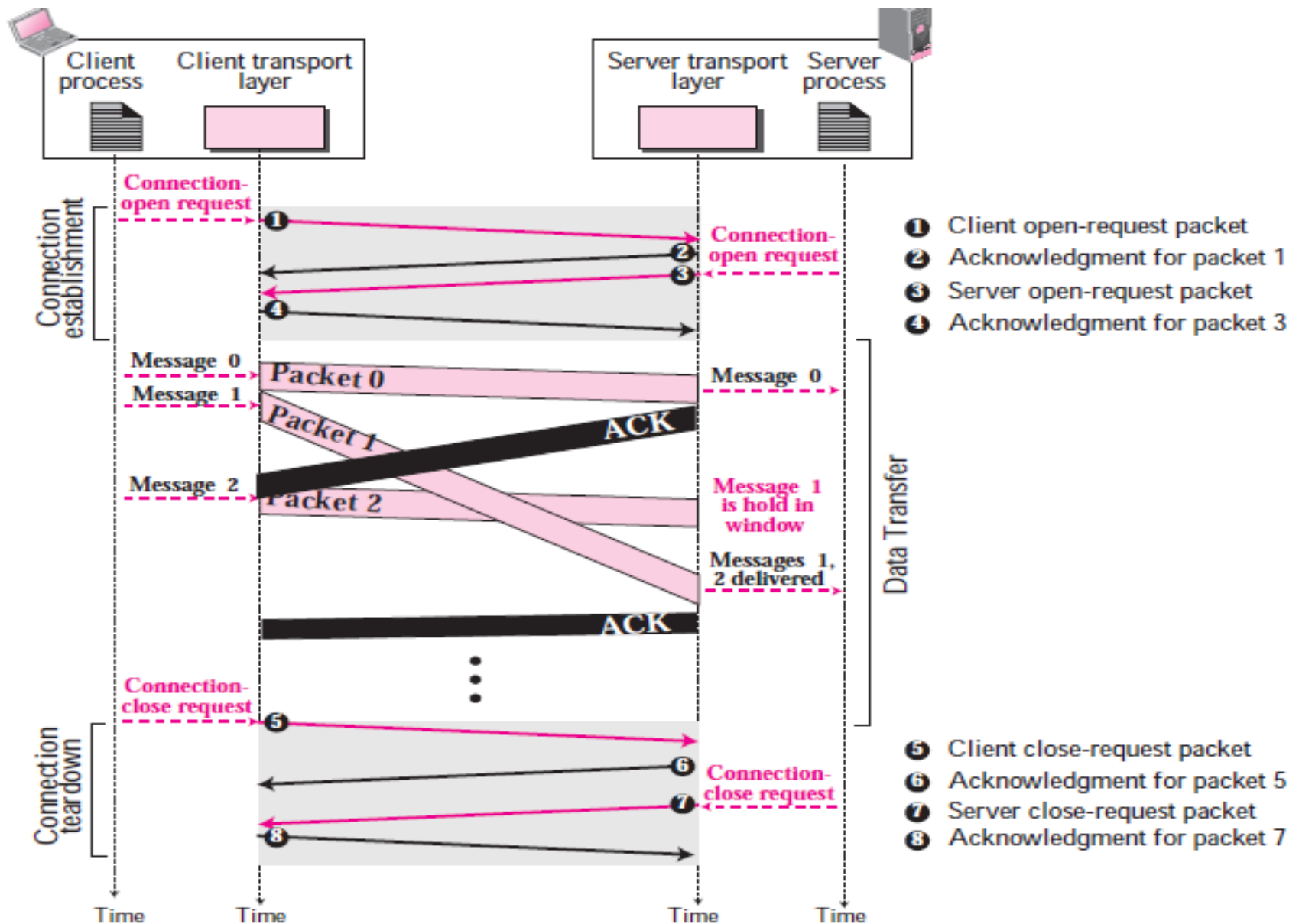


Figure 13.8 *Pushing or pulling*

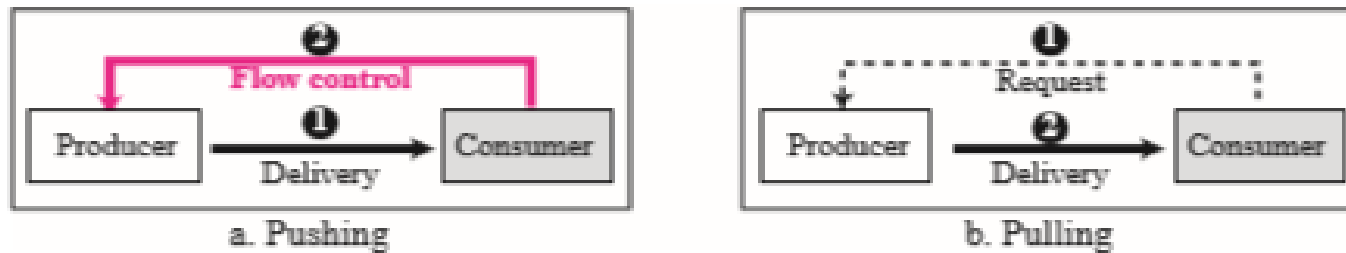
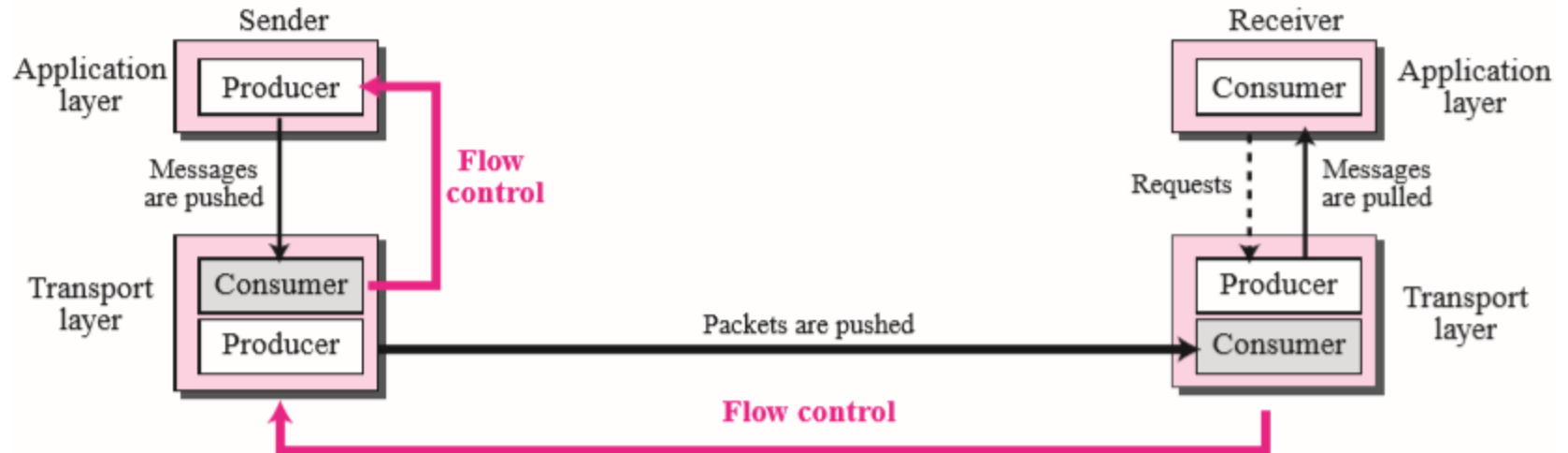
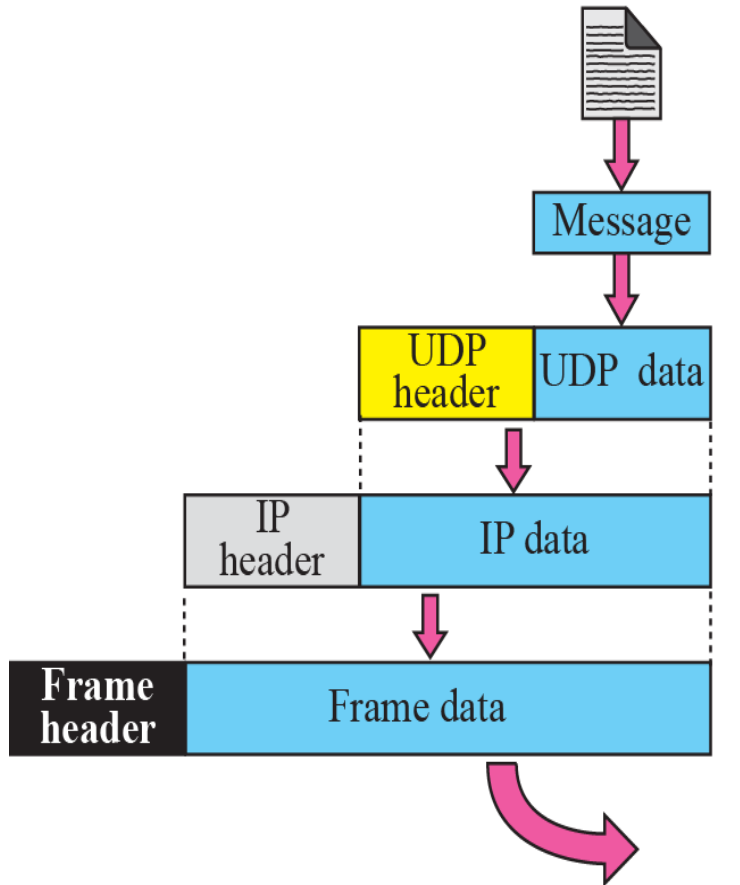


Figure 13.9 *Flow control at the transport layer*



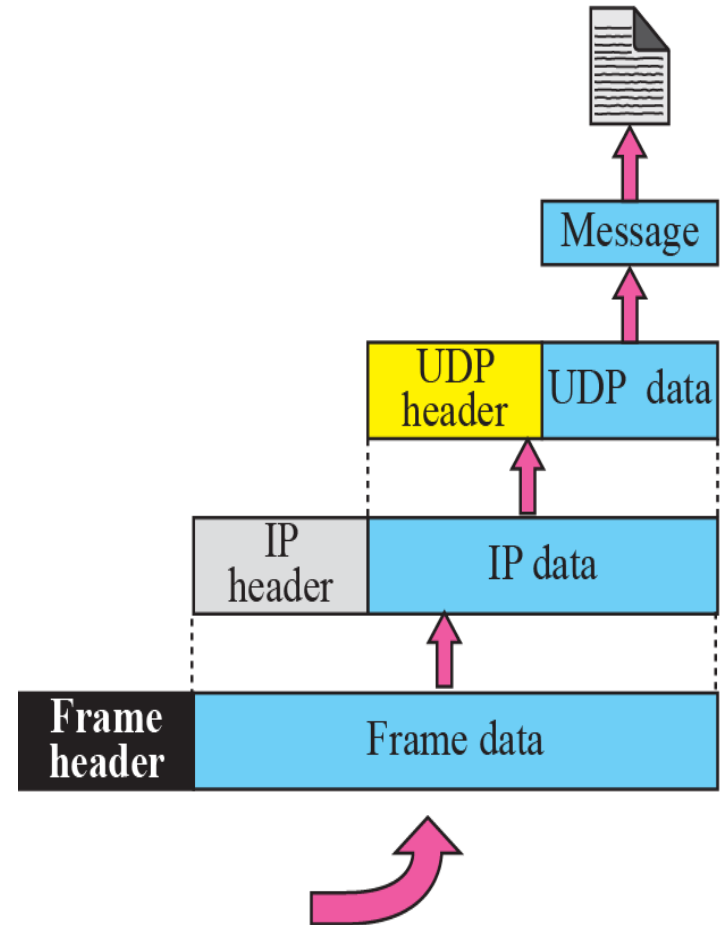
Encapsulation and decapsulation

Sender Process



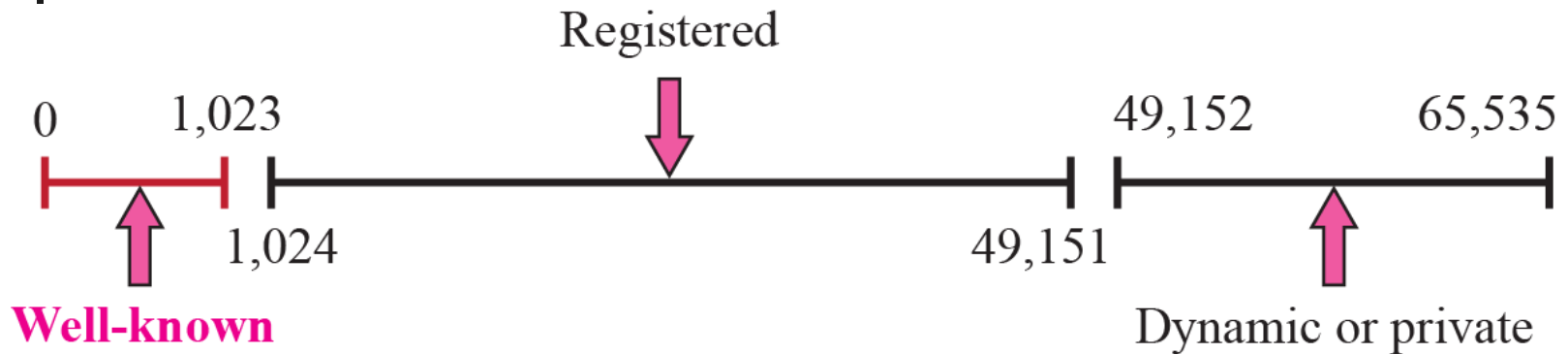
a. Encapsulation

Receiver Process

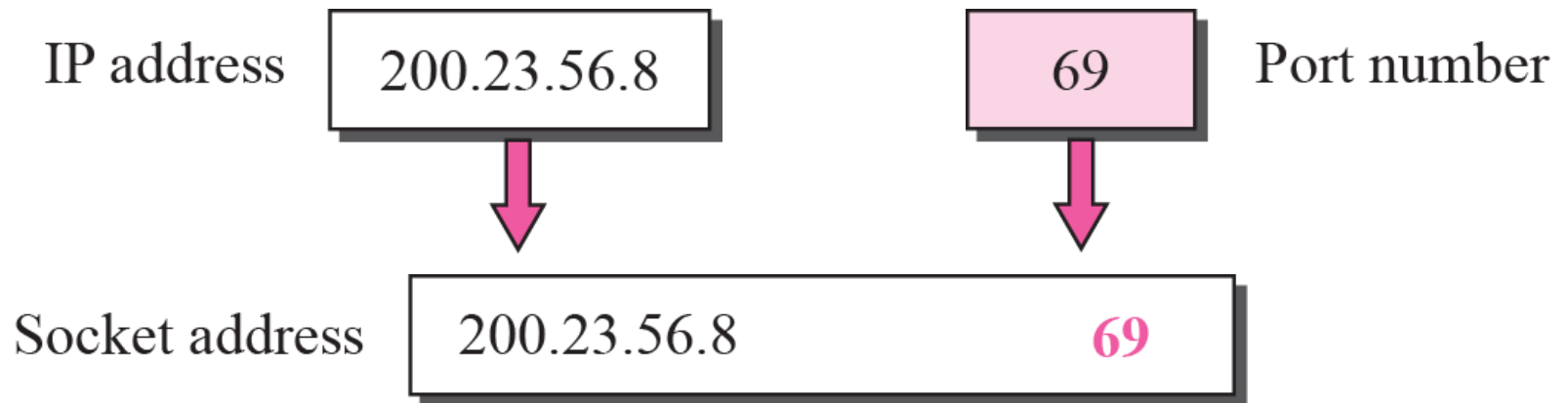


b. Decapsulation

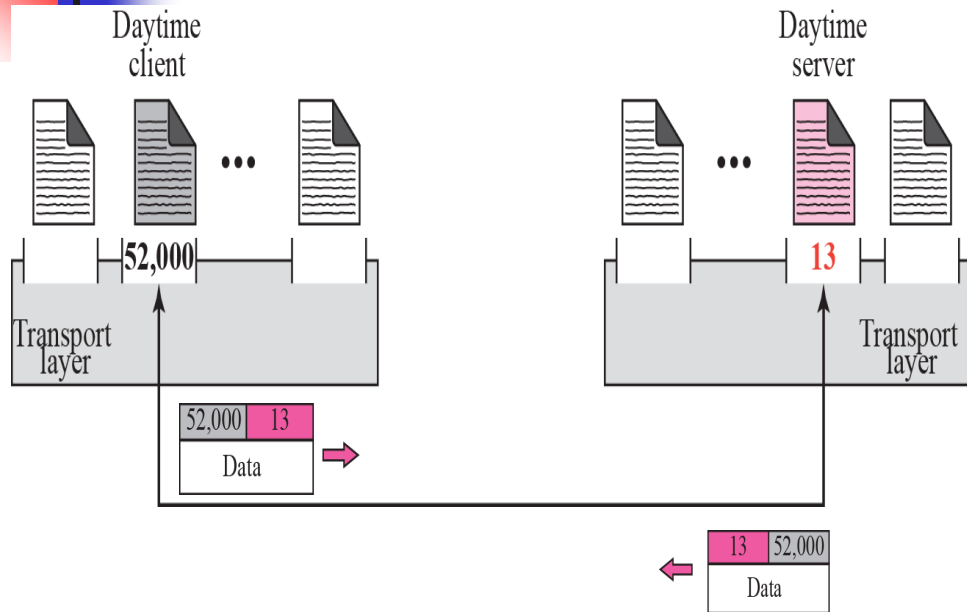
ICANN ranges



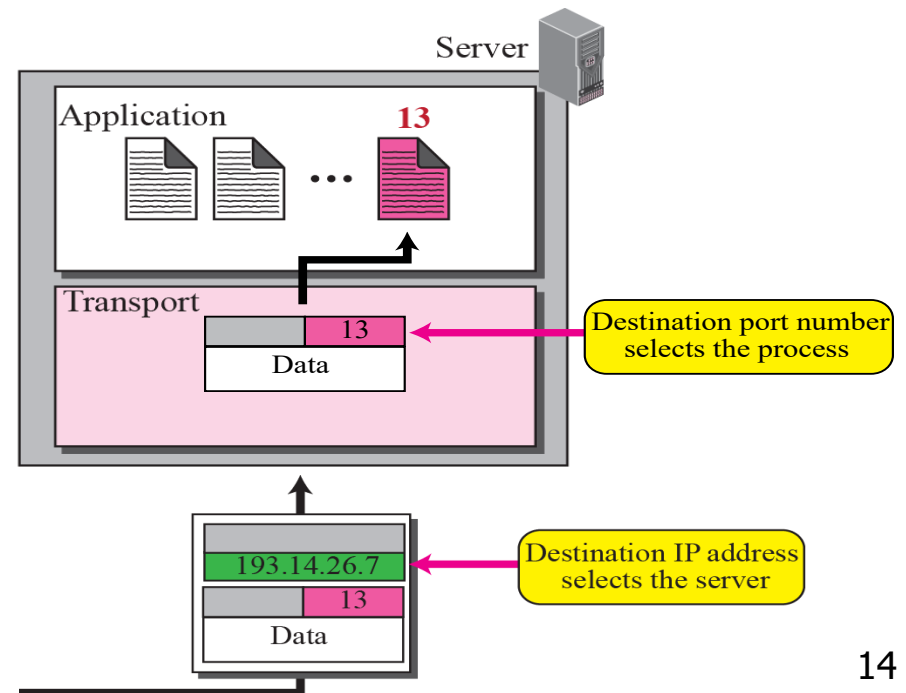
Socket address



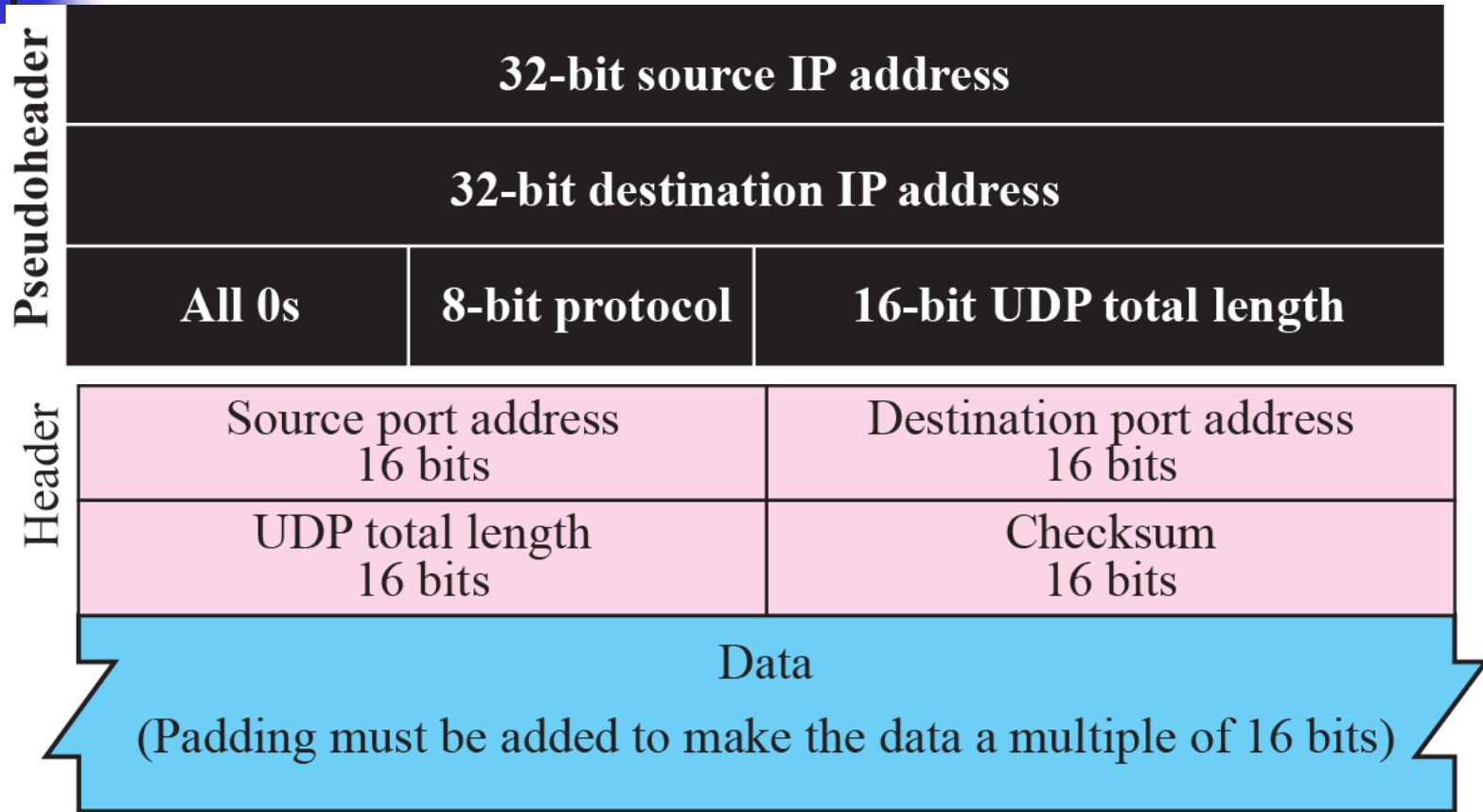
Port numbers



IP addresses versus port numbers



Pseudoheader for checksum calculation



Checksum inclusion is Optional –

In such case, the checksum field is all filled with 0

Checksum calculation for a simple UDP user datagram

Consider a very small user datagram with only 7 bytes of data. Because the number of bytes of data is odd, padding is added for checksum calculation.

The pseudoheader as well as the padding will be dropped when the user datagram is delivered to IP

153.18.8.105			
171.2.14.10			
All 0s	17	15	
1087		13	
15		All 0s	
T	E	S	T
I	N	G	Pad

10011001	00010010	→	153.18
00001000	01101001	→	8.105
10101011	00000010	→	171.2
00001110	00001010	→	14.10
00000000	00010001	→	0 and 17
00000000	00001111	→	15
00000100	00111111	→	1087
00000000	00001101	→	13
00000000	00001111	→	15
00000000	00000000	→	0 (checksum)
01010100	01000101	→	T and E
01010011	01010100	→	S and T
01001001	01001110	→	I and N
01000111	00000000	→	G and 0 (padding)
10010110	11101011	→	Sum
01101001	00010100	→	Checksum

Example

What value is sent for the checksum in one of the following hypothetical situations?

- a.** The sender decides not to include the checksum.
- b.** The sender decides to include the checksum, but the value of the sum is all 1s.
- c.** The sender decides to include the checksum, but the value of the sum is all 0s.

Solution

The sender decides not to include the checksum

The value sent for the checksum field is all 0s to show that the checksum is not calculated.

The sender decides to include the checksum, but the value of the sum is all 1s.

When the sender complements the sum, the result is all 0s; the sender complements the result again before sending. The value sent for the checksum is all 1s.

The second complement operation is needed to avoid confusion with the case in part a.

The sender decides to include the checksum, but the value of the sum is all 0s.

This situation never happens because it implies that the value of every term included in the calculation of the sum is all 0s, which is impossible; some fields in the pseudoheader have nonzero values



***UDP is connectionless simple protocol
with an optional checksum added to
packets for error detection.***

**Although UDP meets almost none of the criteria
for a reliable transport-layer protocol,**

UDP is preferable for some applications.

Applications that can benefit more from the services of UDP than from those of TCP

- Suitable for a process that requires simple request-response communication with little concern for flow and error control
- Suitable for process with internal flow and error control mechanism
- Suitable transport protocol for broadcasting
- Used for route updating protocols
- Normally used for real-time applications that can tolerate uneven delay

uneven delay

For example, let us assume that video packets are sent every 30ms. If some of the packets arrive with 30-ms delay and others with 40-ms delay, an uneven quality in the video is the result.

Example

DNS

This client-server application uses the services of UDP because a client needs to send a short request to a server and to receive a quick response from it.

- The request and response can each fit in one user datagram.
- Since only one message is exchanged in each direction, the connectionless feature is not an issue
- the client or server does not worry that messages are delivered out of order.

Example

SMTP *(UDP is not a suitable transport layer)*

This client-server application is used in electronic mail, cannot use the services of UDP

- **user can send a long e-mail message, which may include multimedia (images, audio, or video).**
- ❑ If the application uses UDP and the message does not fit in one single user datagram, the message must be split by the application into different user datagrams.
- ❑ Here the connectionless service may create problems. The user datagrams may arrive and be delivered to the receiver application out of order. The receiver application may not be able to reorder the pieces.

The connectionless service has a disadvantage for an application program that sends long messages.

Downloading a very large text file from the Internet.

(UDP is not a suitable transport layer)

- ✓ We need to use a transport layer that provides reliable service.
- ✓ We don't want part of the file to be missing or corrupted when we open the file.
- ✓ The delay created between the delivery of the parts are not an overriding concern for us; we wait until the whole file is composed before looking at it.

Example

watching a real-time stream video on our computer.

Such a program is considered a long file;

- it is divided into many small parts and broadcast in real time.
- the parts of the message are sent one after another.

If the transport layer is supposed to resend a corrupted or lost frame,

- the synchronizing of the whole transmission may be lost.
- The viewer suddenly sees a blank screen and needs to wait until the second transmission arrives. This is not tolerable.

if each small part of the screen is sent using one single user datagram,

- the receiving UDP can easily ignore the corrupted or lost packet and deliver the rest to the application program. That part of the screen is blank for a very short period of the time, which most viewers do not even notice.

video cannot be viewed out of order,

- so streaming audio, video, and voice applications that run over UDP must reorder or drop frames that are out of sequence.

UDP Services

✓ **Process-to-Process Communication**

It is possible with sockets(combination of IP address and port number)

✓ **Connectionless Service**

No relationship between user datagrams even if they are coming from same source and moving to same destination. Users datagram are not numbered. Therefore only short messages can be send.

✓ **Flow Control** – no flow control mechanism support

✓ **Error Control** – no error control except checksum

✓ **Congestion Control** – no congestion control